Lecture 5.

Brute force approach.

Algorithms and Data Structures
Ivan Solomatin
MIPT

Outline

- Brute force approach.
  - Strings (password brute force)
  - Subsets
  - Permutations
  - Sudoku

Brute force approach.

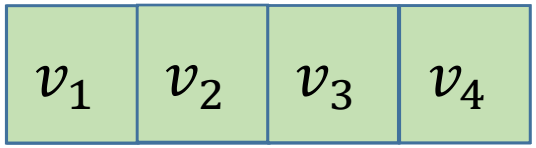
- **Strings (password brute force)**
- Subsets
- Permutations
- Sudoku

Brute force approach.

Strings (password brute force)

Alphabet: A=[a, b, c, d]
Length: 4

Number of permutations:



Number of choices: 4 * 4 * 4 * 4 = 256

Number of permutations: $|A|^N$

aaaa baaa caaa daaa
aaab baab caab daab
aaac baac caac daac
aaad baad caad daad
aaba baba caba daba
aabb babb cabb dabb
aabc babc cabc dabc
aabd babd cabd dabd
aaca baca caca daca
aacb bacb cacb dacb
aacc bacc cacc dacc
aacd bacd cacd dacd
aada bada cada dada
aadb badb cadb dadb
aadc badc cadc dadc
aadd badd cadd dadd
… … … …

Brute force approach.

Strings (password brute force)

```
def strings(x, n, string=None):
    if string is None:
        string = ''
    if n == 0:
        print(string)
    else:
        for v in x:
            strings(x, n - 1, string + v)
```

Brute force approach.

Strings (password brute force)

```
def strings(x, n, string=None):
    if string is None:
        string = ''
    if n == 0:
        yield string
    else:
        for v in x:
            yield from strings(x, n - 1, string + v)

for s in strings('abcd', 4):
    print(s)
```

Brute force approach.

Example

Problem: Password brute force.

There is a password containing lowercase Latin letters and having length $N = 6$.

You have a function check(password), which checks it matches. Find the password.

```
def strings(x, n, string=None):
    if string is None:
        string = ''
    if n == 0:
        yield string
    else:
        for v in x:
            yield from strings(x, n - 1, string + v)

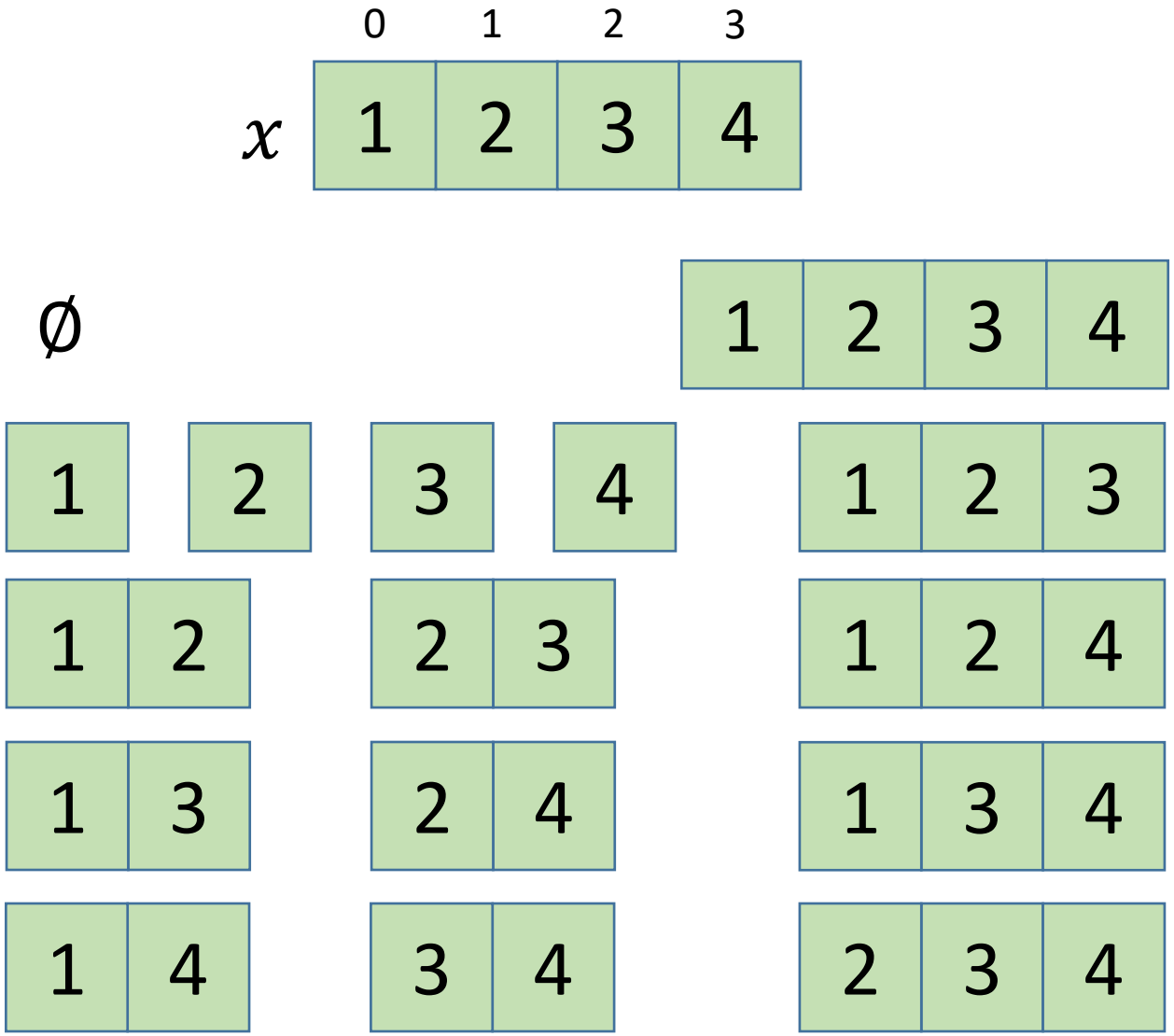
for s in strings('abcdefghijklmnopqrstuvwxyz', 6):
    if check(s):
        print(s)
        break
```

Brute force approach.

- Strings (password brute force)
- **Subsets**
- Permutations
- Sudoku

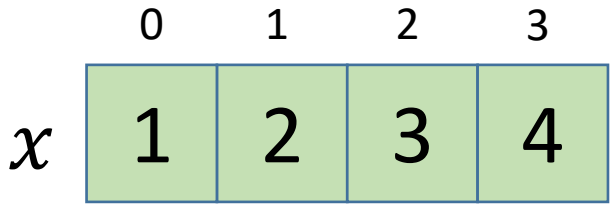
Brute force approach.

Subsets

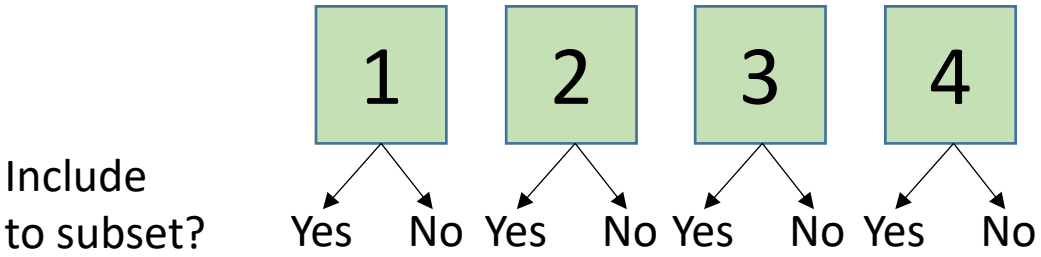


Brute force approach.

Subsets



Number of subsets:



Number of subsets: 2^N

Brute force approach.

Subsets

```
def subsets(x, subset=None):
    if subset is None:
        subset = []
    if len(x) == 0:
        yield subset
    else:
        yield from subsets(x[1:], subset)
        yield from subsets(x[1:], subset + [x[0]])

for s in subsets([1, 2, 3, 4]):
    print(s)
```

Brute force approach.

Example

Problem: Tower.

You want to build a tower of height H. You have N building blocks with heights $h_0, h_1, \ldots h_{N-1}$. Print any combination of blocks which will allow to construct a tower.

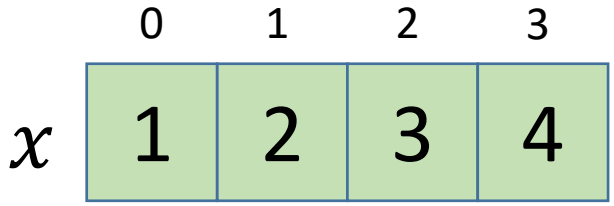
```
for s in subsets([h_0, h_1, ... h_{N-1}]):
    if sum(s) == H:
        print(s)
        break
```

Brute force approach.

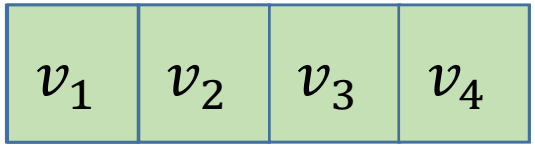
- Strings (password brute force)
- Subsets
- **Permutations**
- Sudoku

Brute force approach.

Permutations



Number of permutations:



Number of choices: 4 * 3 * 2 * 1 = 24

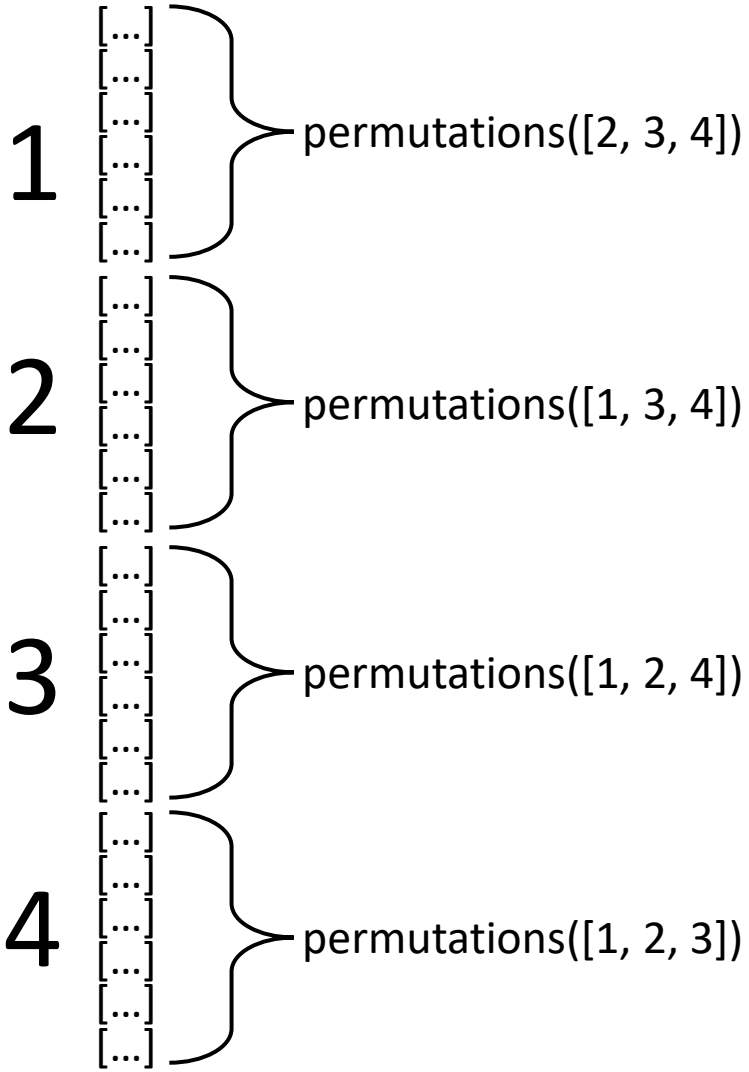
Number of permutations: $N!$

[1, 2, 3, 4]	[3, 1, 2, 4]
[1, 2, 4, 3]	[3, 1, 4, 2]
[1, 3, 2, 4]	[3, 2, 1, 4]
[1, 3, 4, 2]	[3, 2, 4, 1]
[1, 4, 2, 3]	[3, 4, 1, 2]
[1, 4, 3, 2]	[3, 4, 2, 1]
[2, 1, 3, 4]	[4, 1, 2, 3]
[2, 1, 4, 3]	[4, 1, 3, 2]
[2, 3, 1, 4]	[4, 2, 1, 3]
[2, 3, 4, 1]	[4, 2, 3, 1]
[2, 4, 1, 3]	[4, 3, 1, 2]
[2, 4, 3, 1]	[4, 3, 2, 1]

Brute force approach.

Permutations

permutations([1, 2, 3, 4]):



[1, 2, 3, 4] [3, 1, 2, 4]
[1, 2, 4, 3] [3, 1, 4, 2]
[1, 3, 2, 4] [3, 2, 1, 4]
[1, 3, 4, 2] [3, 2, 4, 1]
[1, 4, 2, 3] [3, 4, 1, 2]
[1, 4, 3, 2] [3, 4, 2, 1]
[2, 1, 3, 4] [4, 1, 2, 3]
[2, 1, 4, 3] [4, 1, 3, 2]
[2, 3, 1, 4] [4, 2, 1, 3]
[2, 3, 4, 1] [4, 2, 3, 1]
[2, 4, 1, 3] [4, 3, 1, 2]
[2, 4, 3, 1] [4, 3, 2, 1]

Brute force approach.

Permutations

```
def permutations(x: list, perm=None):
    if perm is None:
        perm = []
    if len(x) == 0:
        yield perm
    for i in range(len(x)):
        yield from permutations(x[:i] + x[i+1:], perm + [x[i]])

for p in permutations([1, 2, 3, 4]):
    print(p)
```

Brute force approach.

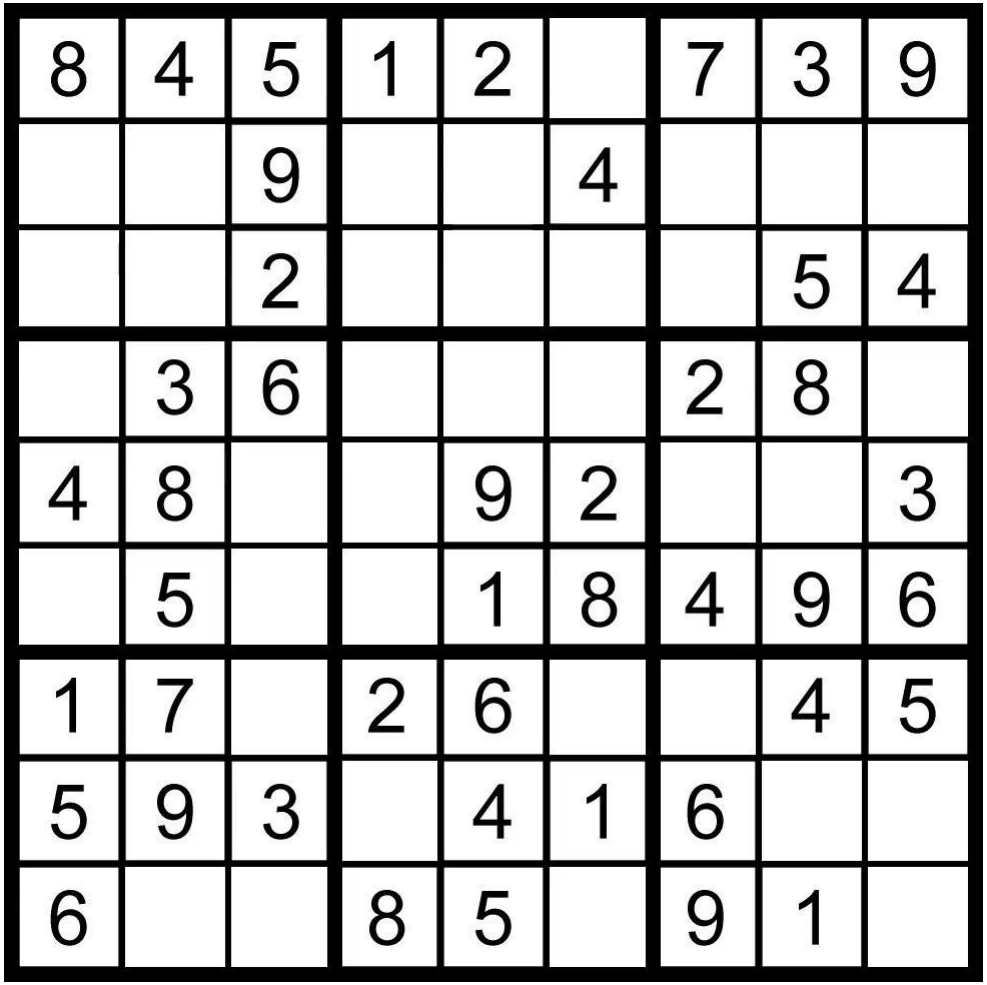
- Strings (password brute force)
- Subsets
- Permutations
- **Sudoku**

Brute force approach.

Sudoku

How can we solve sudoku?

Try all possible values for all empty cells. If value does not fit because of one of the sudoku rules – try the next one.



Brute force approach.

Sudoku

But some choices are incorrect, so we don't need to continue checking values for other empty cells if value of one cell is incorrect.
This is called **pruning** (we reject some possible choices early).

8	4	5	1	2	6	7	3	9
8		9			4			
		2					5	4
	3	6				2	8	
4	8			9	2			3
	5			1	8	4	9	6
1	7		2	6			4	5
5	9	3		4	1	6		
6			8	5		9	1	

Brute force approach.

Sudoku

```
def solve(board):
  for row in range(9):
    for col in range(9):
      if board[row][col] != 0:
        continue
      for digit in range(1, 10):
        if not fits(board, row,
                    col, digit):
          continue
        board[row][col] = digit
        if solve(board):
          return True
        # Otherwise,the choice was
        # incorrect,let's clean it:
        board[row][col] = 0
        # Deeper cells are already cleaned.
      return False
  return True
```

8	4	5	1	2		7	3	9
		9			4			
		2					5	4
	3	6				2	8	
4	8			9	2			3
	5			1	8	4	9	6
1	7		2	6			4	5
5	9	3		4	1	6		
6			8	5		9	1	

Thank you for watching!

Recursion

- **Factorial**
- Hanoi towers
- Practice

Recursion

Factorial

Let's demonstrate idea of recursion by implementing factorial function.

$$n! = 1 * 2 * \cdots * n = \prod_{i=1}^{n} n$$

We can iterate over i=1..n and calculate the product.

But also we can use the following formula:

$$n! = n * (n-1)!$$

Let's implement it:

```
def fact(n):
    return n * fact(n - 1)
```

Are there any issues here?

Recursion

Factorial

This code will go to infinite loop:

```
def fact(n: int):
    return n * fact(n - 1)
```

n will tend to $-\infty$, until stack overflow or `RecursionError: maximum recursion depth exceeded`.
We will discuss it later.

To fix it, we should define not only recursive step, but also stop rule:

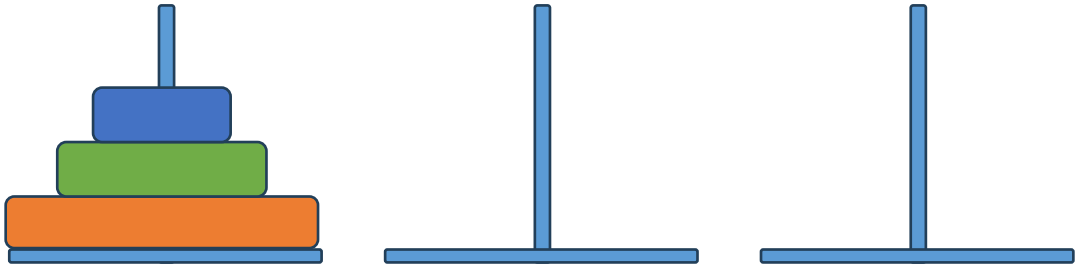
```
def fact(n: int):
    if n < 2:
        return 1
    return n * fact(n - 1)
```

Recursion

- Factorial
- **Hanoi towers**
- Practice

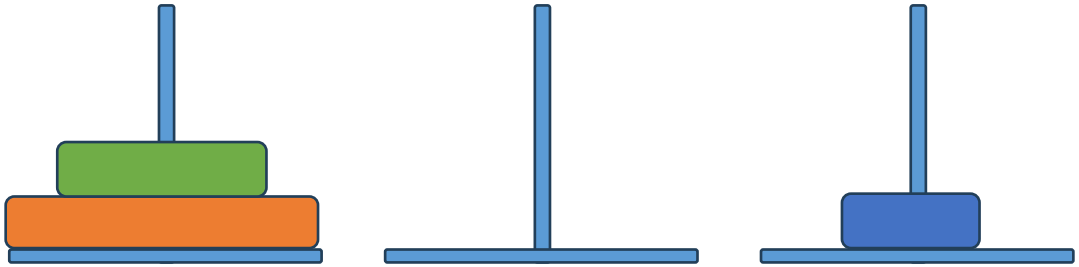
Recursion

Hanoi towers



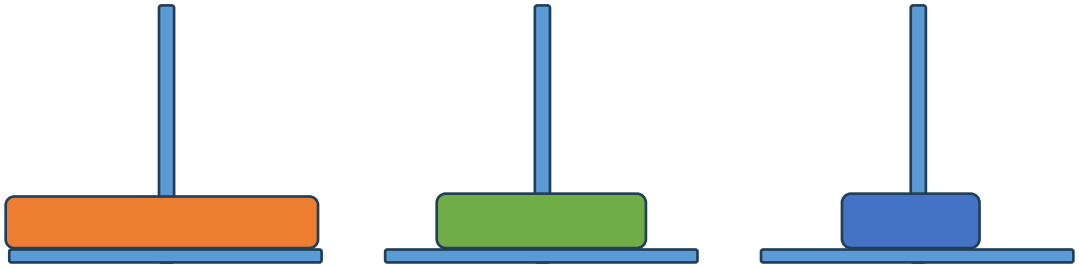
Recursion

Hanoi towers



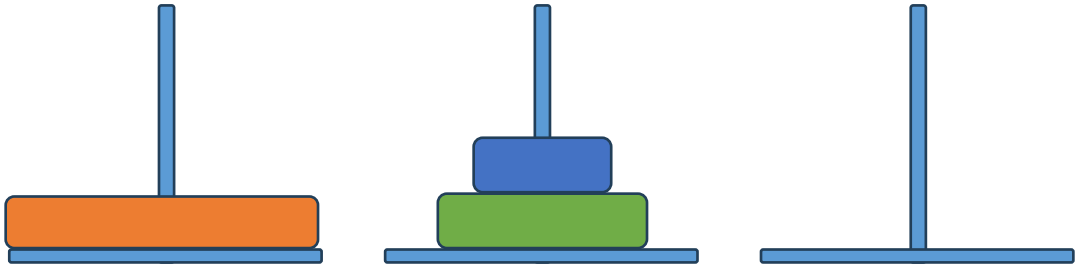
Recursion

Hanoi towers



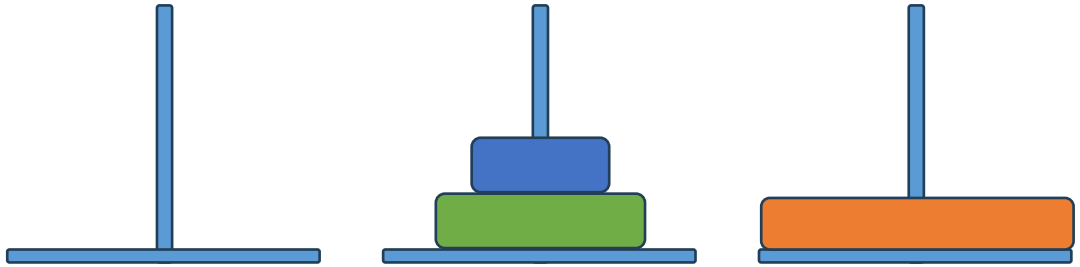
Recursion

Hanoi towers



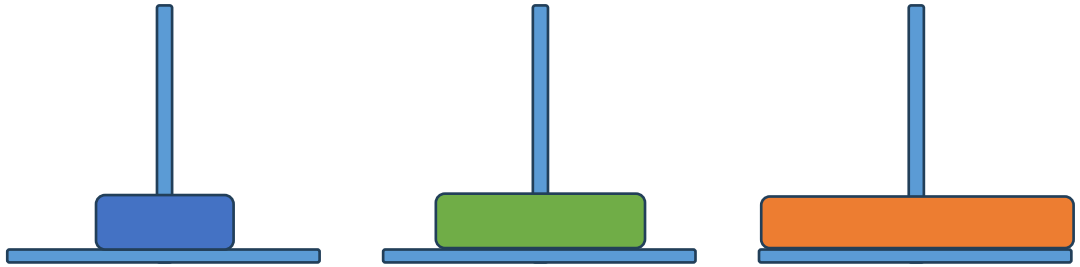
Recursion

Hanoi towers



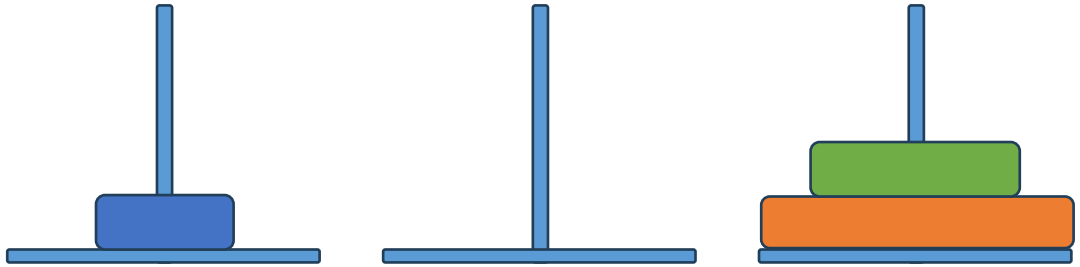
Recursion

Hanoi towers



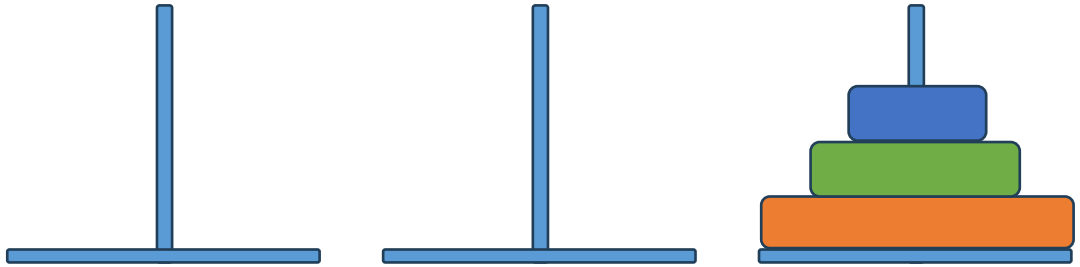
Recursion

Hanoi towers



Recursion

Hanoi towers



Recursion

Hanoi towers

```
def hanoi(n, x, y):
    if n == 1:
        print(1, x, y)
    else:
        hanoi(n - 1, x, 6 - x - y)
        print(n, x, y)
        hanoi(n - 1, 6 - x - y, y)
```